

Chapter 63. Automation Architecture & Governance

From a pile of scripts to one governed platform — priority, tooling, shared services, ownership, and the audit that finds what's actually running.

Approved chapter v1

Approved 20 September 2026

Governs real automations built in Chapters 19, 20, 46, 51 and 58

Contents

Chapter 63. Automation Architecture & Governance

Why this matters

In plain English

63.1 One architecture, not a pile of scripts

63.2 Process discovery and prioritization

63.3 Choosing the right tool for the job

63.4 A reference architecture for automation

63.5 Scheduled versus event-driven design

63.6 Shared services

63.7 Ownership, runbooks, and support

63.8 Controlling sprawl and shadow IT

63.9 Controls: approvals, segregation of duties, audit trails, change management

63.10 Retiring automations safely

Common mistakes and how to spot them

In the real world: the first automation audit

Tools

The project: a design document for Riverstone's reporting and automation platform

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 63. Automation Architecture & Governance

Part VII — Architecture, Governance & Leadership

Chapter at a glance

You will learn to: design the flow from source to delivered result as one architecture instead of a pile of independent scripts · discover and prioritize automation opportunities by value, risk, and effort, with a real worked ROI comparison · choose the right tool for a given job from a full decision matrix — macro, scheduled script, BI subscription, low-code flow, orchestrated pipeline, integration platform, RPA, or an AI agent · apply a reference architecture that every automation in this book turns out to be a specialization of · decide between scheduled and event-driven designs · build the shared services every automation needs rather than reinventing them each time · assign ownership, write runbooks, and support what you've automated · find and control shadow IT before it becomes an unowned, business-critical liability · apply the controls — approvals, segregation of duties, audit trails, change management — that keep automation trustworthy at scale · retire an automation safely.

Before you start: this chapter governs automations built earlier in this book: Chapter 19 (VBA and Apps Script macros), Chapter 20 (the Daily Sales Flash), Chapter 46 (Dagster orchestration, Part V), Chapter 51 (the CRM reverse-ETL sync, Part V), and Chapter 58 (the PO-intake pipeline, Part VI). You don't need to reread any of them — each is reintroduced with the one fact this chapter needs from it.

Time needed: 12–14 hours, spread over a week and a half.

Tools: nothing new — this chapter organizes and governs automations already built with the tools earlier chapters taught.

Practice data: `companion/ch63/` : the full Riverstone automation inventory (24 automations, scored for value, risk, and effort), a center-of-excellence charter template, and a runbook template.

Why this matters

By this point in the book, Riverstone has a lot of automation: a handful of VBA macros holding branch reporting together, a daily email that replaced a manager's morning routine, an orchestrated data pipeline moving real volume, a model reading purchase-order emails, a sync keeping the CRM current. Each one, in its own chapter, was the right answer to the problem in front of it.

Look at all of them together, for the first time, and a different picture appears — the one Chapter 60 first surfaced when Meera drew the whole platform as a diagram and found it "genuinely alarming." Some of these automations have an owner who can explain them in one sentence. Some don't. Some

were built once and never touched again. At least one, per Chapter 19's own story, ran silently wrong for years before anyone checked. Multiply Riverstone's eight or nine well-documented systems by every branch's own macro, every analyst's personal script, every one-off Apps Script project built to solve a Tuesday-afternoon problem and never revisited — and the real number of things quietly running the business is far larger than any diagram anyone has drawn.

This chapter is about that larger number. Not building new automations — Parts II, V, and VI already taught that — but governing the ones that exist: knowing they exist, knowing who owns them, knowing what happens when they break, and having a disciplined way to decide what gets built next and what gets shut down. It's the difference between a company that has automated some things and a company that has an automation *capability* — one that scales, survives people leaving, and doesn't quietly cost more than it saves.

In plain English

Imagine a house where, over ten years, every resident who ever lived there wired in their own convenience: a timer for the porch light, a smart plug for the kettle, a motion sensor in the hallway, a second thermostat someone installed and forgot to register with the first. Each one, on the day it was installed, solved a real problem.

Nobody who lives there now has a full list of what's wired into the walls. Some of it still works perfectly. Some of it fights with something else — the two thermostats arguing over the same room. Some of it is a fire risk nobody's checked in years. And when the power flickers at 2 a.m., the person awake has no idea which of the fifteen anonymous little boxes behind the walls is actually responsible.

Automation architecture and governance is the electrician's audit of that house — not ripping everything out and rewiring from scratch, but finding out what's actually there, who's responsible for each thing, which wiring standard new work has to follow, and which of the old boxes are safe to leave alone versus which need to come out before they start a fire. A company that automates without ever doing this audit ends up with exactly the same problem, at a scale that makes it much more expensive to discover during an actual outage than during a calm Tuesday review.

63.1 One architecture, not a pile of scripts

The starting failure this chapter addresses isn't any single bad automation — it's the *accumulation* of good, reasonable, individually-justified automations into something nobody designed as a whole. Riverstone's real inventory by early 2026 already spans:

- **Chapter 19's branch consolidation macro** and its Apps Script sibling — spreadsheet-bound, built by an analyst, living inside one workbook.
- **Chapter 20's Daily Sales Flash** — a scheduled Python script with its own logging and its own failure alert.

- **Chapter 46's Dagster pipeline** (Part V) — a properly orchestrated system with dependencies, retries, and a real run history.
- **Chapter 51's CRM reverse-ETL sync** (Part V) — a scheduled write-back with its own idempotency and reconciliation logic.
- **Chapter 58's PO-intake pipeline** (Part VI) — an AI-assisted extraction system writing into the ERP.

Each was designed well, in its own chapter, against its own requirements. **None of them was designed with the others in mind**, because at the time each was built, "the others" mostly didn't exist yet. That's not a criticism of any individual chapter's work — it's the normal, expected way automation accumulates in any real company, and it's exactly why a dedicated architectural pass, done deliberately rather than accidentally, matters.

Treating this as one architecture rather than five unrelated projects means asking, for every automation: what shape does it share with the others? Section 63.4's reference architecture answers that question directly — every one of the five above, and every macro built by every branch since, turns out to be a specialization of the same six-layer shape, just with some layers compressed, skipped, or done by hand.

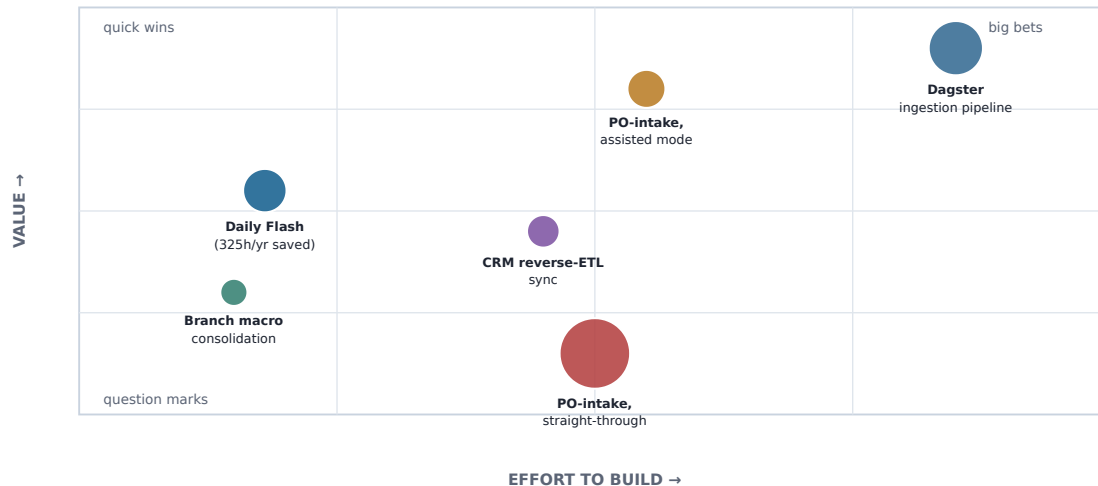
63.2 Process discovery and prioritization

Before building anything new, an architect needs an honest answer to a harder question than "what should we automate next?" — namely, **"of everything that could be automated, what's actually worth it, and what's actually risky?"**

Three factors, scored for every candidate:

- **Value** — time saved, errors prevented, revenue protected or enabled. Chapter 20's own habit (time before, time after, multiplied by frequency) is exactly this, done for one automation; this section does it across many, so they can be compared.
- **Effort** — what it costs to build and, just as importantly, to *operate* afterward. A script that takes a day to write and needs no ongoing attention is cheap; a pipeline that takes a week to write and needs a person watching it every day is not, regardless of the build cost alone.
- **Risk** — what happens when it's wrong, not just when it's down. This is the factor most process-discovery exercises skip, and it's the one that changes the answer most dramatically.

Prioritizing automation: value, effort, and risk, plotted for real



Bubble size is risk (bigger = riskier). Straight-through PO-intake looks cheap and valuable on effort and speed alone — only its risk bubble, drawn honestly, shows why assisted mode (smaller circle, still high value) was the actual choice.

Figure 63.1 — Effort and value alone would rank straight-through PO-intake as a clear win: high value, moderate effort. Its risk bubble — drawn honestly, from Chapter 58's real 19% silent-error finding — is what actually decided against it.

The worked ROI comparison, with real numbers

Automation	Value (₹/yr or time)	Effort to build	Risk if wrong
Daily Sales Flash is caught by (Ch 20) before sending	325 hrs/yr saved (~₹3.9 lakh at loaded cost)	Low (days)	Low — wrong number its own checks
Branch macro consolidation column and (Ch 19) consolidation	~4 min → 9 sec per run, hundreds of runs/year	Low (days)	Low — source_file checks catch a bad
PO-intake, assisted mode confirmation (Ch 58) doesn't ship it wrong	~₹198/day at 40 emails, vs. ₹600/day fully manual	Medium (weeks)	Medium — a missed delays one order,
PO-intake, straight-through error rate, (Ch 58, rejected) order, discovered fact	Looks like ~₹0/day labor cost, but ~₹13,380/day in hidden error cost	Medium (weeks)	HIGH — 19% silent ~₹2,000 per wrong only after the
Dagster ingestion pipeline tested, but a (Ch 46, Part V) dependency	Enables everything downstream; hard to price	High (months)	Medium — well- platform-wide
CRM reverse-ETL sync eventual (Ch 51, Part V) 61); safe to lag	Keeps sales data current, avoids manual re-entry	Medium (weeks)	Low — deliberately consistency (Ch

The finding the table makes obvious, that a value-and-effort-only view would miss entirely: PO-intake's straight-through mode has *better* raw economics than its assisted alternative on labor cost alone — that's precisely why the pilot was tempting. Its risk column is what actually decided the question, and Chapter 58's own conclusion (assisted, not straight-through) is this exact prioritization framework, run for real, landing on the answer its raw ROI number argued against.

A simple, usable prioritization rule: score every candidate on all three factors, plot value against effort (Figure 63.1), and **let the risk bubble's size override a merely-attractive position on the other two axes**. A small, low-risk win (the branch macro) deserves building before a large, high-risk one (straight-through intake) — not because it's more valuable in isolation, but because the *expected* value, once risk is priced in honestly, is genuinely higher.

63.3 Choosing the right tool for the job

Every automation in this book used one of eight tool categories, and Riverstone now has a real example of each — which makes the decision matrix concrete rather than a list of vendor names.

Tool category	Best for	Riverstone's real example	Cost when misused
Spreadsheet macro / script	One workbook, one owner, low volume	Chapter 19's branch consolidation macro	Becomes Chapter 19's nine-year-old unowned single point of failure
Scheduled script	Regular, self-contained jobs with clear inputs and outputs	Chapter 20's Daily Sales Flash	No orchestration, dependency tracking, or shared monitoring at scale
BI subscription / alert	Delivering an existing report or dashboard on a schedule	Chapter 16's Power BI subscriptions	Can't transform data — only delivers what already exists
Low-code / integration flow	Connecting SaaS tools, business-owned, no dedicated engineer	Power Automate / Zoho flows mentioned in Chapter 20	A forty-step flow with no version control, effectively unmaintainable
Orchestrated pipeline	Multi-step, dependency-aware, production data workloads	Chapter 46's Dagster ingestion (Part V)	Overkill — and a real operational burden — for a single weekly report
Integration / iPaaS platform	Enterprise system-to-system sync, vendor-supported	Chapter 51's CRM reverse-ETL pattern	Licensing cost that dwarfs a simpler script for low volume
RPA (robotic process automation)	A legacy system with no API, screen-based only	<i>Not yet used at Riverstone</i> — the honest gap below	Fragile: breaks on every UI change, expensive to maintain
AI agent / LLM pipeline	Unstructured input (email, documents), judgment-shaped tasks	Chapter 58's PO-intake extraction, Chapter 55's RAG assistant	Confident, wrong answers if not paired with human confirmation and refusal logic

The one honest gap in Riverstone's toolkit is RPA, and naming where it *would* fit is more useful than pretending it's irrelevant: `DATA_SPEC.md`'s note about Kolkata's legacy billing system — screen-based, no API, still requiring a spreadsheet upload workaround — is exactly the kind of system RPA exists for. Riverstone hasn't needed it yet because that system's volume is still low enough for a manual workaround; the moment that volume grows, RPA (rather than trying to force the problem into a script that has nothing to call) becomes the right tool, and knowing that in advance is what a decision matrix is for.

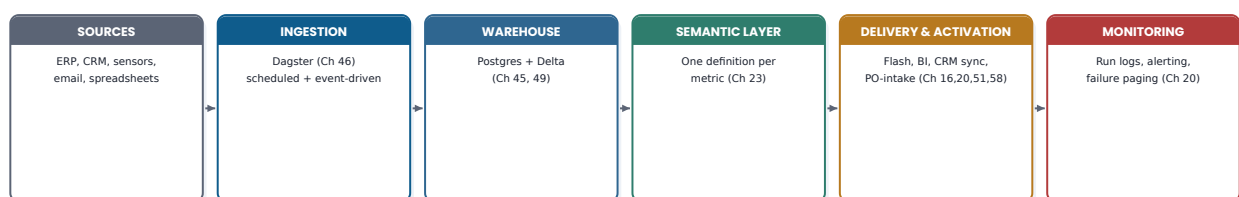
The choice, in practice, comes down to three questions, asked in order:

1. **Does the source system have an API?** If not, and screen automation is the only option, that's RPA's specific niche — not a reason to avoid automating, but a reason to pick the tool built for exactly that constraint.
2. **Is the logic mostly structured transformation, or does it require judgment on unstructured input?** Structured favors a script or pipeline; judgment on emails, documents, or images favors an AI-agent pattern, paired with human confirmation exactly as Chapter 58 settled on.
3. **Who owns it, and what's their skill set?** A flow a business team can maintain themselves (low-code) beats a beautifully engineered pipeline nobody on that team can touch — Chapter 60's fit-over-fashion principle, applied to tooling rather than architecture.

63.4 A reference architecture for automation

Every automation examined so far, however different they look, turns out to share one underlying shape: data comes from somewhere, gets moved and cleaned, gets organized around business meaning, gets delivered or acted on, and someone finds out whether it worked.

A reference architecture for automation, one layer at a time



Every automation in this book is a specialization of this one shape.

A single macro (Ch 19) compresses all six layers into one spreadsheet; a full pipeline (Ch 46) spans all six explicitly.

Figure 63.2 — Every automation in this book is a specialization of this one shape. Recognizing the layer something sits in is the first step to governing it properly.

- **Sources:** the ERP, the CRM, plant sensors, inbound email, a spreadsheet someone maintains by hand.
- **Ingestion:** getting data out of a source and into somewhere it can be worked with — Chapter 46's Dagster jobs, or a macro's `Workbooks.Open` call doing the same job at a smaller scale.
- **Warehouse:** where it lands and is organized for reuse — Chapters 45 and 49's Postgres and Delta Lake tables, or, for a macro, the `Master` worksheet it builds each run.

- **Semantic layer:** one agreed meaning per metric — Chapter 23's models, or, for a small automation, simply the one formula the macro's author decided "revenue" means.
- **Delivery and activation:** where the result actually reaches someone or does something — the Flash email, a dashboard, the CRM write, the ERP order.
- **Monitoring:** how anyone finds out whether it worked — Chapter 20's checks and failure alerts, or, for an ungoverned macro, nothing at all, which is precisely the gap section 63.8 addresses.

Why naming the layers matters more than it first appears: a macro that compresses all six layers into one Excel workbook isn't doing something fundamentally different from Chapter 46's full pipeline — it's doing the same six jobs, just informally, invisibly, and without any of the shared services (section 63.6) a properly governed automation gets for free. Recognizing "this macro is quietly doing ingestion, warehousing, and delivery all in one VBA module" is the first step toward deciding whether that's still the right shape for it, or whether it's outgrown the workbook it lives in.

63.5 Scheduled versus event-driven design

Two fundamentally different rhythms govern when an automation runs, and Chapter 61's PACELC reasoning applies directly to choosing between them.

Scheduled automations run at a fixed time — Chapter 20's Flash at 07:30, Chapter 46's nightly ingestion. They're simple to reason about, easy to debug (the run either happened at the expected time or it didn't), and they accept a bounded staleness: whatever happened between runs isn't reflected until the next one fires.

Event-driven automations run in response to something happening — Chapter 58's PO-intake pipeline reacting to an arriving email, a webhook firing the moment a CRM record changes. They're more responsive (Chapter 61's latency side of the trade), and they're genuinely more complex to build correctly: they need to handle bursts (many events at once), gaps (an event that never arrives), and duplicates (the same event delivered twice) — exactly the message-delivery guarantees Chapter 61's reliability toolkit exists to provide.

The choice is the same latency-versus-complexity trade-off from Chapter 61's PACELC, applied to timing specifically:

Choose scheduled when...	Choose event-driven when...
A daily or weekly cadence genuinely matches how the business uses the result	Being minutes late has a real, specific cost (a line alert, a time-sensitive order)
Batching many records together is more efficient than handling them one at a time	Each event is naturally independent and small
The team's operational maturity favors the simpler failure mode	The team has (or is building) the infrastructure to handle bursts, gaps, and duplicates safely

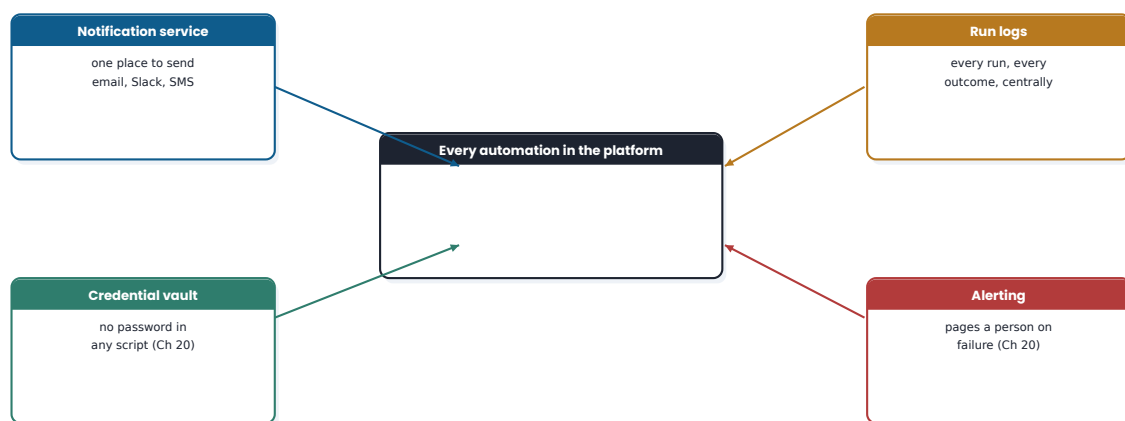
Riverstone's own platform uses both deliberately: the Flash is scheduled because a sales manager checking revenue once a day is a completely adequate cadence; the defect-detection line alert (Chapter 56) is event-driven because a delay of even a few seconds defeats its purpose. **Defaulting to**

event-driven everywhere because it sounds more sophisticated is a real, common mistake — most business reporting genuinely doesn't need sub-second freshness, and the added complexity of an event-driven design, paid for that occasion, buys nothing anyone asked for.

63.6 Shared services

Every automation needs to send a notification, keep a credential somewhere, log what it did, and alert someone when it fails. Building these four things once, well, and sharing them across every automation is one of the highest-leverage investments an automation platform can make.

Shared services: build once, every automation uses them



Without these built once and shared, every one of Riverstone's 40+ automations reinvents its own logging, its own alerting, and — worst of all — its own place to hide a password. Chapter 20's daily flash already assumes all four exist.

Figure 63.3 — Build these four once. Chapter 20's Daily Flash already assumes all four exist; the goal of this section is making that assumption true platform-wide, not just for one well-built script.

- **Notification service:** one place that knows how to send an email, a Slack message, an SMS — so every automation calls the same function instead of forty automations each embedding their own SMTP configuration, forty of which will eventually break the same way when the mail server's settings change.
- **Credential vault:** no automation stores a password, API key, or connection string in its own code. Chapter 20's environment-variable discipline is the minimum version of this; a real vault (a managed secrets service) is the version that scales past a handful of scripts and lets credentials be rotated without editing code anywhere.
- **Run logs:** every automation's every run — start time, end time, outcome, key numbers — written to one place, not scattered across forty different log files in forty different formats. This is what makes Chapter 63.8's audit possible at all: you cannot inventory what never left a trace.
- **Alerting:** one shared mechanism for "something needs a human," so failures page the right person consistently, rather than each automation inventing its own notion of who to tell and how urgently.

The payoff compounds with every automation added. The fifth automation built on top of these four shared services costs a fraction of what the first one cost to build safely, because logging, alerting, and credential handling are already solved problems by then — exactly the leverage a platform is supposed to provide, and exactly what a pile of independent scripts never gets.

63.7 Ownership, runbooks, and support

An automation without a named owner is not really automated — it's a piece of infrastructure the company depends on that nobody is accountable for, which is precisely Chapter 19's `MASTER_FINAL_v7_USE_THIS.xlsm` and precisely the situation section 63.8's audit exists to find and fix.

Ownership means one person (or one team) can answer, without research, three questions: what does this do, why does it do it that way, and what happens if it breaks? Chapter 20's one-page handover note (section 20.13) is the minimum viable version of this, and it scales: the same seven headings — what it does, when it runs, inputs, outputs, checks, failure playbook, owner and deputy — work whether the automation is a five-line script or a company-wide pipeline.

A runbook goes one step further than a handover note: it's the specific, step-by-step instructions for the three or four most likely failures, written so that someone who is *not* the original builder can follow them at 2 a.m. without improvising. "Refresh failed: the ERP load ran late. Rerun after 07:30 with `python daily_flash.py <date> --send` " (Chapter 20's own example) is a runbook entry in miniature — concrete, copy-pasteable, and written before the failure happens, not reconstructed from memory during it.

Support doesn't end at handover. Every automation needs, at minimum: a named deputy who has actually run it once (not just read about it), a review date on the handover note so it doesn't silently go stale, and an honest answer to "what happens if the owner is unreachable and it fails right now?" An automation that fails that last question isn't ready to be relied on, however well it was built.

63.8 Controlling sprawl and shadow IT

Shadow IT is any automation running the business that the people responsible for the company's systems don't know exists. It isn't the work of careless people — Chapter 19's story is the honest, sympathetic version: an analyst, years ago, solved a real Tuesday-afternoon problem with a macro, it worked, it kept working, and by the time anyone thought to ask "who owns this and does it still work correctly," the person who wrote it had left the company.

The first step is not a policy — it's an inventory. You cannot govern what you haven't found.

The first automation audit: what an inventory actually finds

Category	Count	Examples
Documented, owned, monitored	6	Ch 20 Flash, Ch 46 pipeline, Ch 51 sync, Ch 55 assistant, Ch 56/57 model services, Ch 58 PO-int...
Working, but no owner on record	11	Branch-level macros, personal Apps Script projects, one Power Automate flow nobody remembers bu...
Duplicated logic, disagreeing outputs	4	Two branches independently built near-identical consolidation macros with different bugs
Business-critical, single point of failure	1	MASTER_FINAL_v7_USE_THIS.xlsm (Ch 19) — still running, still unowned until the Ch 19 rewrite
Actively broken, nobody had noticed	2	A stale Apps Script trigger silently stopped firing 5 months earlier

24 automations found, once someone actually looked. Only 6 had ever been through anything like this chapter's governance model.

Figure 63.4 — Twenty-four automations found, once someone actually looked for them. Eighteen of them had never been through anything resembling this chapter's governance model.

Running an inventory, in practice: ask every team what spreadsheet, script, or flow they rely on weekly. Check every branch's shared drive for `.xlsm` files. Check the company's low-code platform (Power Automate, Zoho Flow) for anything created more than a year ago. Check for scheduled tasks and cron entries on any machine the data team can access. None of this is exotic detective work — it's simply the first time anyone has asked the question company-wide rather than team-by-team.

What Riverstone's own first audit found, and what each category demands:

- **Documented, owned, monitored (6):** the automations this book has already governed properly — leave them alone, they're the model to replicate.
- **Working, but no owner on record (11):** the largest category, and the most urgent — not because they're broken, but because nobody would notice if they became broken. Each needs an owner assigned this quarter, using the handover-note template from section 63.7.
- **Duplicated logic, disagreeing outputs (4):** two branches had each independently built a near-identical consolidation macro, with different bugs, producing different numbers for what should be the same calculation — precisely Chapter 62's "two domains, two definitions" failure mode (section 62.5), discovered here in spreadsheet form rather than warehouse form.
- **Business-critical, single point of failure (1):** Chapter 19's macro itself, which by the time of this audit had already been through its own rewrite — the audit's job here is confirming the fix held, not repeating it.
- **Actively broken, nobody had noticed (2):** the most sobering finding. A stale Apps Script trigger had silently stopped firing five months before anyone checked, and nothing downstream had noticed its absence — which says as much about the downstream consumer's own lack of freshness monitoring (Chapter 47) as it does about the broken trigger itself.

Standards, established once the inventory exists: every new automation gets a named owner before it goes live, not after; every new automation uses the shared services from section 63.6 rather than reinventing them; every automation above a stated risk threshold (section 63.2's scoring) goes through the controls in section 63.9 before deployment.

A center of excellence is the ongoing home for these standards — not a gatekeeper that approves every macro anyone wants to write, but a small group (often just the platform team, formalized) that maintains the inventory, publishes the standards, offers the shared services as something easy to adopt rather than a mandate to fight, and runs this audit again on a schedule rather than once and

never again. The center of excellence succeeds when building the *right* way is the *easy* way — when a new analyst reaches for the shared logging function because it's less work than writing their own, not because a policy document told them to.

63.9 Controls: approvals, segregation of duties, audit trails, change management

Not every automation needs every control in this section — a personal script that reformats a spreadsheet needs none of them. **An automation that moves money, changes a customer record, or makes a decision on someone's behalf needs all of them**, and the judgment of which category something falls into is itself part of the architect's job.

- **Approvals:** a defined threshold above which an automation needs a human sign-off before it acts. Chapter 58's PO-intake pipeline uses exactly this — Riverstone's ₹100,000 approval limit means anything above it routes to a person, regardless of how confident the model is.
- **Segregation of duties:** the person who builds an automation shouldn't be the only person who can approve its changes going live, particularly for anything touching money or customer data. This isn't distrust of the builder — it's the same principle behind a second pair of eyes on any consequential decision, and it's what catches a mistake the builder is too close to their own work to see.
- **Audit trails:** a record of what an automation did, when, and — for anything that writes to another system — on whose authority. Chapter 51's idempotency keys serve double duty here: they prevent duplicate writes *and* provide exactly the trail an auditor would ask for.
- **Change management:** a real process, however lightweight, before a production automation's logic changes — at minimum, someone other than the author looking at the change, and a way to roll it back if it goes wrong. Chapter 60's ADRs are change management for architectural decisions; a code review is the equivalent for an automation's day-to-day logic.

The right amount of control scales with the risk, not with the size of the automation. A five-line script that writes financial transactions needs real controls despite its size; a thousand-line report generator that only reads data and emails a PDF needs almost none, despite its length. Applying uniform, heavy process to everything is how governance earns a reputation for slowing teams down for no benefit — and how people quietly route around it, recreating exactly the shadow IT problem section 63.8 exists to fix.

63.10 Retiring automations safely

Every automation eventually outlives its usefulness — the report it produces stops being read, the process it supports gets redesigned, a better system replaces it. **Retiring one safely is not simply deleting a file**, and skipping this step is how "nobody knows if it's safe to turn off" becomes a permanent reason never to turn anything off.

A safe retirement checklist:

1. **Confirm nothing downstream depends on it.** Check the run logs (section 63.6) for who's actually consuming the output, not who was originally meant to.
2. **Announce a sunset date**, publicly, with enough lead time for anyone quietly depending on it to speak up.
3. **Turn it off in stages where possible** — stop the schedule but leave the code and its last output in place for a defined period, rather than deleting everything the same day.
4. **Archive, don't delete**, the code and its documentation for a reasonable retention period — the next audit (section 63.8) should be able to find a retired automation's history, not just its absence.
5. **Update the inventory** the same day, so the center of excellence's record of what's running stays accurate.

The alternative — automations that nobody dares to turn off because nobody's sure what depends on them — is its own form of sprawl, arguably a more expensive one than an unowned macro, because it actively consumes maintenance effort for a benefit nobody can name. **A platform that can retire things safely is what makes it safe to build new things aggressively:** teams experiment more freely when they trust that yesterday's experiment can be cleanly removed if it doesn't pan out.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Building automations with no shared shape in mind	Forty scripts, each reinventing logging, alerting, and credential handling	Recognize the six-layer reference architecture in every new automation
Prioritizing by value and effort alone	A risky automation looks like the best investment on paper	Score risk honestly, and let a large risk bubble override an attractive position on the other two axes
Choosing a tool by familiarity rather than fit	An orchestrated pipeline built for a single weekly report; a fragile RPA script forced onto a system with a perfectly good API	Work through the decision matrix's three questions before picking a tool
Defaulting to event-driven because it sounds more sophisticated	Real complexity paid for a freshness requirement nobody asked for	Default to scheduled; justify event-driven with a specific, named cost of delay
No shared services	Every automation stores its own credentials, logs its own way, alerts nobody	Build notification, vault, logging, and alerting once, and make them the easy default
No named owner	Nobody can explain what an automation does or what happens if it breaks	Ownership assigned before anything goes live, using a real handover note
A handover note nobody's tested	The deputy has read about the process but never run it	Have the deputy actually run it once before calling the handover complete
Governing only what's already known about	Shadow IT accumulates invisibly for years	Run a real, company-wide inventory — ask every team, check every drive

Mistake	Symptom	Fix
Uniform, heavy controls on everything	Low-risk automations slowed down for no benefit; people route around governance entirely	Scale controls to risk, not to an automation's size or visibility
Never retiring anything	Maintenance effort spent on automations nobody uses, "just in case"	A defined, staged retirement process, with the inventory updated the same day
Treating the center of excellence as a gatekeeper	Teams avoid it, and shadow IT returns	Make the governed way the easy way — shared services people want to use
Confusing a policy document with actual governance	Standards exist on paper; nothing in production actually follows them	An inventory that's checked, not just a document that's written

In the real world: the first automation audit

In April 2026, two months into her role as Head of Data Platform (Chapter 60), Meera Iyer did something Riverstone had never formally done: she asked every team, in writing, one question — *"What spreadsheet, script, or automated flow does your work depend on weekly, that isn't one of the platform team's own systems?"*

She expected a handful of answers. She got twenty-four.

Eleven of them were automations nobody on her team had ever heard of: a Bengaluru branch macro that reformatted weekly stock counts, a personal Apps Script project one analyst had built to auto-file expense receipts, a Power Automate flow connecting the CRM to a WhatsApp group that three people depended on and nobody could name the original author of. Four were duplicated: Delhi and Kolkata had each, independently, built their own version of Chapter 19's consolidation macro, with subtly different bugs, and neither branch knew the other's existed. Two were actively broken — most strikingly, an Apps Script trigger that had silently stopped firing five months earlier, discovered only because the audit asked "when did you last check that this actually ran?" and nobody could answer.

The genuinely uncomfortable finding wasn't any single broken thing. It was the shape of the whole list: **eighteen of twenty-four automations Riverstone depended on weekly had never been through anything resembling ownership, monitoring, or a documented failure plan.** The company's real automation footprint was three times the size of what Meera's own platform team had built and knew about.

Her response wasn't to shut anything down. Every one of the twenty-four was, by definition, solving a real problem someone depended on — the alternative to a slightly risky automation is not "no automation," it's "back to the manual process that was slow enough to justify building the automation in the first place." Instead, over the following quarter:

1. **Every automation got a named owner**, using the section 63.7 handover template — including the ones nobody had built, where "owner" meant whoever currently depended on it most, formally accepting that responsibility going forward.

2. **The four duplicated macros were consolidated into one**, owned by the platform team, replacing both branches' versions — Chapter 19's original rewrite, done a second time, at company scale.
3. **The two broken automations were fixed**, and — more importantly — both were migrated onto the shared monitoring service (section 63.6), so the *next* silent failure would page someone within hours, not go unnoticed for months.
4. **A standing inventory review was scheduled quarterly**, not as a one-time project but as an ongoing habit — the center of excellence's actual, ongoing job, rather than a document written once and never revisited.

Nine months later, at the next scheduled review, the count had grown to twenty-nine automations — genuine growth, not sprawl, because every new one arrived with an owner and a handover note from day one, the direct result of the standards section 63.8 describes.

What made the difference:

- **She asked the question company-wide, not team-by-team**, which is the only way to find automation nobody thought to mention because it had quietly become invisible.
- **She treated every finding as a gap to close, not a mistake to punish** — which is why teams kept answering honestly in the next round, rather than learning to hide what they'd built.
- **She fixed the structural problem (no shared monitoring, no ownership standard), not just the two symptoms** she happened to find — the difference between governance and firefighting.
- **The audit became a habit, not a project** — the only way an inventory stays true is if someone keeps asking the question.

Tools

- No new software this chapter — every tool named in section 63.3's decision matrix was introduced earlier in this book.
 - **Companion files (`companion/ch63/`):**
 - `automation-inventory.csv` : the full 24-automation inventory behind Figure 63.4, with type, owner, value/risk/effort scores, and status for each.
 - `center-of-excellence-charter.md` : a template for the standing group and its responsibilities, from section 63.8.
 - `runbook-template.md` : a blank runbook, expanding Chapter 20's handover note with the step-by-step failure playbook section 63.7 describes.
-

The project: a design document for Riverstone's reporting and automation platform

Goal: replace 40 manual reports, legacy VBA macros, Apps Script projects, and scattered scripts with one governed platform — a design document in Chapter 60's format, applied to automation specifically.

Option A: your own workplace. Run the section 63.8 inventory exercise for real: ask every team what they depend on weekly that isn't a known, owned system.

Option B: Riverstone. Use `companion/ch63/automation-inventory.csv` as your starting inventory.

Steps

1. **Score every automation in your inventory** for value, effort, and risk (section 63.2), and plot them the way Figure 63.1 does.
2. **Classify each one against the reference architecture** (section 63.4): which layers does it cover, and which does it skip or do informally?
3. **For the five highest-risk, lowest-governance automations**, propose the right tool (section 63.3) if it were rebuilt today, and say whether it should be rebuilt at all or simply given an owner and left alone.
4. **Design the shared services** (section 63.6) your platform needs, and identify which existing automations could adopt them with the least effort — the quick wins that prove the platform's value early.
5. **Write one real handover note and one real runbook** (section 63.7) for your single most business-critical, least-documented automation.
6. **Propose the controls** (section 63.9) appropriate to your three highest-risk automations specifically — not a blanket policy for everything.
7. **Write the center-of-excellence charter** (section 63.8): who's in it, what it owns, how often the inventory gets re-run.
8. **State what gets retired**, if anything, using section 63.10's checklist, and what specifically would need to be true before it's safe to do so.

What good looks like: the inventory has at least one honestly uncomfortable finding (an unowned, business-critical automation, or a duplicated one); the risk scoring changes at least one prioritization decision from what value-and-effort alone would suggest; the handover note and runbook are specific enough that someone who didn't build the automation could actually use them.

Stretch goals

- Run the section 63.8 inventory question for real, on your own team or organization, and compare what you find against Riverstone's 24-automation result.
- Design the specific migration path for one shadow-IT automation into the governed platform, following Chapter 60's evolve-under-constraints pattern (migrate the lowest-risk piece first, run old and new in parallel).

- Calculate the total hours-saved-per-year across your full inventory, the way Chapter 20's section 20.14 did for one automation, and use it to make the business case for the platform investment this project proposes.
-

You've got it when...

- ☐ You can look at any automation and place it on the six-layer reference architecture, even if it compresses several layers into one.
 - ☐ You score value, effort, and risk separately, and let a large risk finding override an otherwise-attractive prioritization.
 - ☐ You choose a tool from the full decision matrix based on fit — API availability, structured versus judgment-based logic, who has to maintain it — not familiarity.
 - ☐ You default to scheduled automation and can name the specific cost of delay that would justify event-driven instead.
 - ☐ You build shared services once rather than letting every automation reinvent logging, alerting, and credential handling.
 - ☐ Every automation you're responsible for has a named owner, a handover note, and a runbook for its most likely failures.
 - ☐ You've run, or would know how to run, a real company-wide automation inventory — not just a review of what your own team built.
 - ☐ You scale controls (approvals, segregation of duties, audit trails, change management) to risk, not to visibility or size.
 - ☐ You can retire an automation safely, with a real checklist, rather than either never touching it again or deleting it blind.
-

Recap

- **Automation accumulates into an architecture whether anyone designs it that way or not** — Riverstone's real inventory spans macros, scheduled scripts, orchestrated pipelines, and AI-assisted systems, each built well individually and never designed together.
- **Prioritize by value, effort, and risk together** — Chapter 58's real PO-intake numbers show a high-value, moderate-effort option (straight-through) losing to a lower-value one (assisted) purely on honestly-scored risk.
- **Choose the tool from a real decision matrix:** macro, scheduled script, BI subscription, low-code flow, orchestrated pipeline, integration platform, RPA, or an AI agent — each with a genuine Riverstone example, including the honest gap (RPA, not yet needed).
- **Every automation is a specialization of one reference architecture:** sources, ingestion, warehouse, semantic layer, delivery and activation, monitoring — recognizing which layers an automation covers, and which it skips, is the first governance step.

- **Choose scheduled by default; justify event-driven with a specific cost of delay** — Chapter 61's PACELC reasoning, applied to timing.
 - **Shared services — notification, credential vault, run logs, alerting — built once, save every automation built afterward from reinventing them.**
 - **Ownership, a handover note, and a tested runbook** are what separate an automation from a liability with no name attached.
 - **Shadow IT is found, not assumed away:** a real, company-wide inventory at Riverstone found eighteen of twenty-four weekly-depended-on automations with no ownership, monitoring, or documented failure plan.
 - **Controls scale with risk, not size:** approvals, segregation of duties, audit trails, and change management matter enormously for anything moving money or changing customer data, and barely at all for a read-only report.
 - **Retirement is a process, not a deletion** — confirm nothing depends on it, announce a sunset, stage the shutdown, archive rather than delete, and update the inventory the same day.
-

Practice exercises

Warm-up

1. Name the six layers of the reference architecture, in order, and give a one-sentence example of each from your own experience (work or personal).
2. Why does risk deserve its own axis in prioritization, separate from value and effort? Give an example where ignoring it would produce the wrong decision.
3. What's the difference between a handover note and a runbook?
4. Name three of the eight tool categories from the decision matrix, and a situation where each is clearly the right choice.
5. Why is "we'll delete it if it turns out nobody needs it" not a safe way to retire an automation?

Core

6. Using Chapter 58's real numbers (section 63.2), explain in your own words why straight-through PO-intake looked attractive on value and effort alone, and what specifically the risk score changed about the decision.
7. Classify Chapter 20's Daily Sales Flash against the six-layer reference architecture: which layers does it cover explicitly, and how?
8. A stakeholder asks for a new automation to sync data between two internal tools with good APIs, judged as high-risk because it touches customer records. Walk through the three tool-selection questions (section 63.3) and the controls (section 63.9) it would need.
9. Design the four shared services (section 63.6) for a team of your own: what would the notification service, credential vault, run logs, and alerting actually look like at your current scale?

- l10. Write a one-page handover note (Chapter 20's seven-heading template) for an automation you personally rely on, even an informal one.
- l11. Riverstone's audit found four duplicated macros. Using Chapter 62's data-contract reasoning (section 62.5), explain why "each branch owns its own macro" without a shared definition produced this exact failure.
- l12. A automation candidate scores high value, low effort, and low risk. Another scores high value, low effort, and high risk. Using Figure 63.1's method, explain how you'd sequence building them.

Stretch

- l13. Design the full migration plan for consolidating Riverstone's four duplicated branch macros into one governed automation, following Chapter 60's evolve-under-constraints pattern from section 60.6.
- l14. Propose Riverstone's RPA use case (the Kolkata legacy billing system) as a real project: what would trigger building it, and what would the decision matrix say about the tool choice once that trigger arrives?
- l15. Using the section 63.9 controls, design the approval and audit-trail requirements for a hypothetical automation that adjusts customer credit limits automatically based on payment history.

Think about it

- l16. Is there a size of company below which formal automation governance (an inventory, a center of excellence, standing controls) is genuinely not worth the overhead? Where's that line, and what would move it?
- l17. A team resists adopting the shared services (section 63.6) because their own ad-hoc version "already works fine." How do you make the case for migrating, without simply mandating it?

Key terms

automation architecture · process discovery · value/risk/effort prioritization · tool decision matrix · RPA (robotic process automation) · reference architecture · sources / ingestion / warehouse / semantic layer / delivery and activation / monitoring · scheduled automation · event-driven automation · shared services · notification service · credential vault · run log · alerting · ownership · handover note · runbook · shadow IT · automation inventory · center of excellence · approvals · segregation of duties · audit trail · change management · safe retirement

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 19 and 20:** the specific automations this chapter governs, revisited from a company-wide rather than single-automation view.
 - **Chapter 60, Designing Whole Systems:** the container diagram and shared-services band this chapter's reference architecture directly extends.
 - **Chapter 61, Distributed Systems & Trade-offs:** the scheduled-versus-event-driven choice applies PACELC directly; the reliability toolkit underlies section 63.6's shared services.
 - **Chapter 62, Data Architecture Patterns:** the duplicated-macro finding in this chapter's story is section 62.5's "no contract, no shared definition" failure mode, found in spreadsheets rather than warehouse tables.
 - **Chapter 64, Security, Privacy, Governance & Responsible AI:** the controls in section 63.9 (approvals, audit trails) are this chapter's entry point into a much fuller governance and compliance treatment.
 - **Interview preparation:** the Architecture & Leadership Question Bank asks directly about automation governance and shadow IT — "how would you find out what's actually running in a company you just joined" is this chapter's method, asked as an interview question.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Sources (the ERP, a spreadsheet you maintain by hand); ingestion (a script that pulls data out of that source); warehouse (wherever it's organized for reuse — a database table, or just a cleaned-up worksheet); semantic layer (the one agreed meaning of a term you use, even informally); delivery and activation (an email, a chart, a written-back record); monitoring (how you'd find out if it stopped working — even if today the honest answer is "I'd notice eventually").
2. Value and effort together only tell you whether something is attractive to build; risk tells you what happens when it's wrong. Chapter 58's straight-through PO-intake example is exactly this: it scores well on value and effort, and ignoring risk would mean shipping something with a 19% silent error rate, each error costing roughly ₹2,000 to unwind — a mistake the value/effort view alone would never catch.
3. A handover note is the general orientation document — what an automation does, when it runs, its inputs and outputs, and who owns it. A runbook is narrower and more operational: specific, step-by-step instructions for the most likely failures, written so someone who isn't the original builder can follow them under pressure.
4. For example: a scheduled script for a self-contained daily report with no complex dependencies; an orchestrated pipeline for a multi-step production data workload with real dependency chains; RPA for a legacy system with no API and only a screen-based interface to automate against.

5. Because "nobody needs it" is often only true until the one time a year someone does — a month-end process, an annual report, an edge case that only occurs during a specific season. Deleting without confirming, announcing, and staging risks losing something whose only visible dependency shows up rarely, exactly when it would be most disruptive to discover the loss.
6. Straight-through PO-intake looked attractive because it eliminates the ~45 seconds of human confirmation time per order, appearing to reduce labor cost to nearly zero at high volume — real value, for real, low effort once the extraction pipeline already exists. The risk score changed the decision by pricing in what value/effort ignored: a 19% silent error rate at ₹2,000 per wrong order works out to roughly ₹13,380/day in hidden cost at 40 emails/day — far more expensive than the labor it was meant to save, once the errors are actually counted rather than assumed away.
7. Sources: the ERP's order data. Ingestion and warehouse: the SQL query pulling the day's and trend data directly (a lightweight version, not a full staged warehouse load). Semantic layer: the headline calculations (revenue, orders, AOV) computed consistently. Delivery and activation: the HTML email itself. Monitoring: the four checks that can stop the send, plus the failure alert — Chapter 20's Flash is unusually complete against this framework precisely because it was built with exactly these concerns in mind from the start.
8. Tool selection: (1) both tools have good APIs, so this favors an integration platform or a well-built scheduled script over RPA; (2) the logic is structured (moving records between two systems), favoring a script or integration platform over an AI-agent pattern; (3) who maintains it decides between a low-code flow (if the business team should own it) and an engineered pipeline (if the data platform team should). Controls: because it touches customer records, it needs an audit trail at minimum, and likely approval thresholds and segregation of duties given the risk classification stated in the request.
9. Personal exercise; check that all four services are addressed concretely (not "we'll figure it out") and sized to the team's actual current scale rather than an aspirational one.
10. Personal exercise; check the note follows the seven Chapter 20 headings and could genuinely be understood by someone who didn't build the automation.
11. Without a shared, agreed definition of what the consolidation macro should produce (Chapter 62's data contract, applied here to a spreadsheet rather than a warehouse table), each branch's analyst made their own reasonable, locally-consistent choices about how to build it — and those choices diverged, exactly as two domains publishing their own definition of "active_customer" diverged in Chapter 62's example. The fix is the same in both cases: one agreed, shared, owned version, not two independently-maintained ones.
12. Build the high-value/low-effort/low-risk one first — it's a clear win on every axis with nothing to override the obvious call. Build the high-value/low-effort/high-risk one second, but only after either reducing the risk (a smaller pilot, added human confirmation) or explicitly accepting it with the right controls (section 63.9) in place — the same reasoning Chapter 58 applied when it chose assisted mode over straight-through rather than abandoning automation on that risky candidate entirely.
13. Following section 60.6: (1) pick the lower-risk of the two existing macros as the baseline rather than starting from scratch; (2) run the consolidated version alongside both existing macros for one full reporting cycle, comparing outputs; (3) migrate one branch first, keep the other branch's existing

macro live as a fallback until the new one has proven itself; (4) write the ADR recording the decision, what was rejected (keeping both branch versions, building a third new one from scratch), and why; (5) retire both original macros only once the consolidated version has run cleanly through a full cycle.

14. The trigger would be the Kolkata legacy billing system's transaction volume crossing the point where the manual spreadsheet-upload workaround becomes a real bottleneck or a real error source — not a fixed date, but an observed cost, the same kind of concrete trigger Chapter 62's mesh-readiness discussion used. Once that trigger arrives, the decision matrix's first question (does the source have an API?) answers itself — no — which points directly at RPA as the right tool, rather than trying to force a script to call an API that doesn't exist.

15. Approval: any credit-limit increase above a defined threshold routes to a person before taking effect, mirroring Chapter 58's ₹100,000 rule; smaller adjustments within a conservative band might auto-apply. Audit trail: every adjustment logged with the customer, the old and new limit, the payment-history data that drove the decision, and a timestamp — auditable both for a customer dispute and for confirming the automation itself is behaving as intended. Segregation of duties: the person who built the credit-scoring logic shouldn't be the sole approver of changes to its thresholds going forward.

16. There's no fixed headcount threshold — the right trigger is the same one Chapter 62 used for data mesh readiness: when the number of automations, or the cost of one going wrong, grows past what one or two people can hold in their heads without a formal inventory. A two-person team with three scripts genuinely doesn't need a center of excellence; the same two-person team with thirty scripts, several touching money, does — the governance investment should track the actual risk and scale, not a company's headcount alone.

17. Show, don't mandate: build the shared service well enough that adopting it is genuinely less work than maintaining their own version — easier logging, alerting that actually pages someone reliably, credential handling that survives a password rotation without editing code. Migrate one of their existing automations onto it as a demonstration, with their cooperation, rather than requiring migration of everything at once; a team that sees their own ad-hoc pain solved by the shared version adopts the rest voluntarily, which is a far more durable outcome than a mandate that gets quietly worked around.